QML Lab • research project 11

Quantum Reservoir Computing for Time Series

A fixed random quantum system as a reservoir, with only a linear read-out trained.

Living document • last updated October 25, 2026

Quadrant	CQ (classical data, quantum processor)	**Track**	Hybrid
Difficulty	Medium	**Tools**	PennyLane, scikit-learn
Builds on	Modules 2 and 4 of the QML Lab	**Workspace**	`qml_projects/quantum-reservoir-time-series/`

Summary. Reservoir computing avoids training the hard part of a recurrent model: a fixed dynamical system mixes the input history, and only a linear read-out is fitted. This project uses a small, fixed quantum circuit as the reservoir, feeds a time series into it step by step, and forecasts the series from measured expectation values. It compares the quantum reservoir with a classical echo-state network of the same size on standard forecasting and memory tasks.

1 Background and key concepts

1.1 Reservoir computing

An echo-state network (Jaeger and Haas, 2004) drives a fixed random recurrent network with the input u_t and fits only the output weights. With features $x_t \in \mathbb{R}^m$ collected in a matrix X and targets Y, the read-out is ridge regression,

$$W = YX^\top \left(XX^\top + \lambda I\right)^{-1}, \qquad \hat{y}_t = W x_t, \tag{1}$$

which trains in closed form. The reservoir needs *fading memory*: recent inputs must matter more than old ones.

1.2 Quantum reservoirs

Fujii and Nakajima (2017) proposed quantum reservoirs: the input is injected into one qubit at every step, the system evolves under a fixed Hamiltonian, and expectation values form the features. In a circuit version,

$$\rho_{t+1} = \mathcal{U}\Big(\, |u_t\rangle\langle u_t| \otimes \mathrm{Tr}_1[\rho_t] \Big)\, \mathcal{U}^\dagger, \qquad x_t = \big(\langle Z_i\rangle_t,\ \langle Z_i Z_j\rangle_t\big)_{i<j}, \tag{2}$$

where $|u_t\rangle = R_Y(u_t)\,|0\rangle$ replaces qubit 1 at each step (the partial reset gives fading memory). The exponentially large state space promises rich dynamics from few qubits; the review of Mujal et al. (2021) discusses the opportunities and the cost of measurement.

1.3 Benchmarks

The Mackey–Glass system (Mackey and Glass, 1977), $\dot{x}(t) = \beta\, x(t-\tau)/\left(1 + x(t-\tau)^{10}\right) - \gamma x(t)$ with $\beta = 0.2$, $\gamma = 0.1$ and $\tau = 17$, is chaotic and a standard forecasting test. Memory capacity $\mathrm{MC} = \sum_{d\geq 1} r^2\big(u_{t-d}, \hat{y}_t^{(d)}\big)$ measures how many past inputs the reservoir can reconstruct.

2 Research questions

The project is open-ended: these questions are the starting point, not the finish line. Add new ones to the README and the journal as they come up.

RQ1 How does forecast error depend on reservoir size and on which observables are measured?

RQ2 Does memory of past inputs fade too fast or too slowly, and how can the reset strength tune it?

RQ3 How does the quantum reservoir compare with a classical echo-state network with the same number of features?

3 Method

3.1 Data

A noisy sine wave (sanity check), the Mackey–Glass series sampled at unit time steps, and i.i.d. uniform inputs for memory capacity. 100 washout steps, 1000 training steps, 500 test steps.

3.2 Models

$n = 4$–8 qubits simulated as density matrices (`default.mixed`); fixed random rotations and CNOTs as $\mathcal{U}$; input injection by reset and $R_Y(u_t)$ on qubit 1; single- and two-qubit Z expectation values as features (exact, then from M shots); ridge read-out.

3.3 Baselines

Echo-state network with the same number of read-out features; linear autoregressive model; a small LSTM as a strong upper reference.

3.4 Metrics

- Normalised root mean squared error (NRMSE) on test data.
- Memory capacity curve r^2 versus delay d.
- Robustness to the number of shots M.

4 Planned experiments

Each experiment becomes one `[[experiments]]` entry in the workspace, so its runs, parameters and metrics are stored and compared in the portal.

5 Running it in the QML Lab

The workspace lives in `qml_projects/quantum-reservoir-time-series/`. Scripts import three helpers from `qml_lab/plots.py`:

ID	Experiment	What varies	Main metrics
E1	Sine forecast	reservoir size n	NRMSE
E2	Mackey–Glass	quantum vs echo-state network	NRMSE one- and multi-step
E3	Features	single vs two-qubit observables	NRMSE
E4	Memory	delay $d = 1..20$	memory capacity
E5	Shot noise	$M \in \{100, 1000, 10^4\}$	NRMSE

Table 1: Experiment plan. Results go into the journal and the results log (section 9).

- `params()` reads the parameters from the portal's run form (defaults when run by hand);
- `save()` stores figures with the run;
- `log_metrics()` records numbers that appear in the run table.

```
qml_projects/quantum-reservoir-time-series/
  project.toml          metadata + experiments
  README.md             goal, open questions, ideas
  experiments/          runnable scripts
  docs/main.tex         this document
```

First experiment: This experiment is *planned*: write the script, add the block below to `project.toml`, then run `python manage.py sync_projects`.

```
[[experiments]]
slug = "mackey-glass"
title = "Quantum reservoir forecast of Mackey--Glass"
script = "experiments/e01_mackey_glass.py"
figure = "forecast.png"

[experiments.params]
n_qubits = 6
washout = 100
train_steps = 1000
ridge = 1e-06
shots = 0
task = "mackey_glass"
seed = 0
```

```
# from the portal folder: pick up changes to project.toml
python manage.py sync_projects
cd qml_projects/quantum-reservoir-time-series
# run by hand with the default parameters
PYTHONPATH=../../qml_lab python experiments/e01_mackey_glass.py
```

6 Quality bar

- A classical baseline with a similar number of trainable parameters, tuned with the same care (Bowles et al., 2024).
- At least five random seeds per configuration, reported as mean ± standard deviation.

- How results change with the number of qubits.
- At least one run under a realistic noise model.
- Give the echo-state network exactly as many read-out features as the quantum reservoir.

7 Open directions

- Closed-loop (generative) forecasting with feedback.
- Quantum extreme learning machines, which drop the memory and process one input at a time.
- Hardware with mid-circuit measurement and reset.

8 Risks and limitations

- On hardware, measuring destroys the reservoir state; each time step may require re-running the input history.
- For $n \leq 8$ qubits the reservoir is easy to simulate classically; any advantage must come from scaling.
- Results depend strongly on reservoir and ridge hyperparameters.

9 Results log

Summarise findings here as they become stable; day-to-day notes belong in the portal journal.

Date	Run	Finding	Next step

References

Bowles, J., Ahmed, S. and Schuld, M. (2024) 'Better than classical? The subtle art of benchmarking quantum machine learning models', Preprint, arXiv:2403.07059. Available at: https://arxiv.org/abs/2403.07059.

Fujii, K. and Nakajima, K. (2017) 'Harnessing disordered-ensemble quantum dynamics for machine learning', *Physical Review Applied*, 8(2), 024030. Available at: https://doi.org/10.1103/PhysRevApplied.8.024030.

Jaeger, H. and Haas, H. (2004) 'Harnessing nonlinearity: Predicting chaotic systems and saving energy in wireless communication', *Science*, 304(5667), pp. 78–80. Available at: https://doi.org/10.1126/science.1091277.

Mackey, M. C. and Glass, L. (1977) 'Oscillation and chaos in physiological control systems', *Science*, 197(4300), pp. 287–289. Available at: https://doi.org/10.1126/science.267326.

Mujal, P., Martínez-Peña, R., Nokkala, J., García-Beni, J., Giorgi, G. L., Soriano, M. C. and Zambrini, R. (2021) 'Opportunities in quantum reservoir computing and extreme learning machines', *Advanced Quantum Technologies*, 4(8), 2100027. Available at: https://doi.org/10.1002/qute.202100027.